Functional Programming – Lazy Evaluation

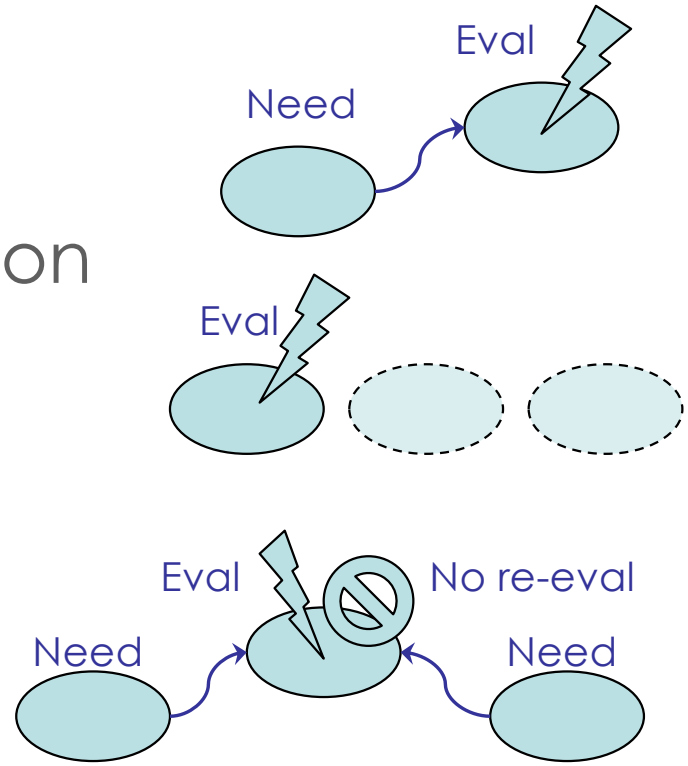
James Gain
Department of Computer Science
University of Cape Town
jgain@cs.uct.ac.za





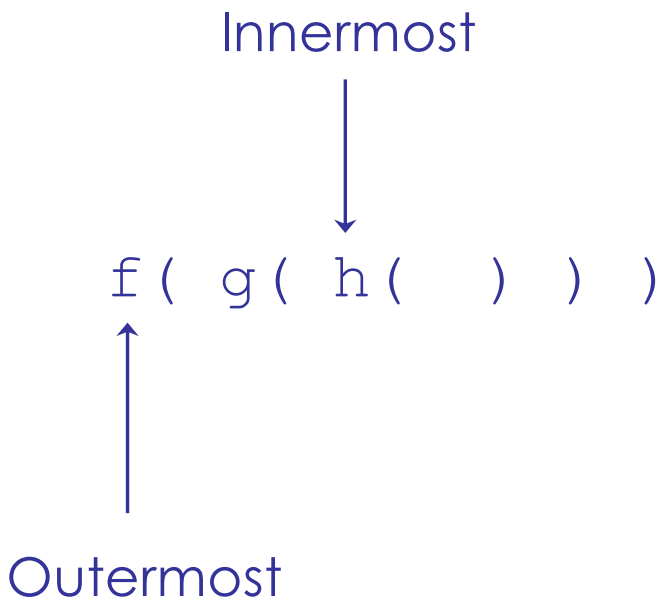
Lazy Evaluation Defined

- Procrastination as an execution philosophy
- Avoid doing unnecessary evaluation (by delaying as long as possible)
  - Evaluate only when needed
  - Evaluate only enough
  - Evaluate at most once
- Allows coding with infinite lists



Evaluating Expressions

- Expressions are evaluated (or reduced) by applying definitions until no further simplification is possible
- Two strategies for deciding which redex (reducible expression) to choose
  - Innermost - always choose the innermost redex of a nested expression
  - Outermost – work from the outermost redex inwards



Redex Features

- Assuming pure functions
- Termination:
  - Outermost has best termination behaviour
  - Strictly: If ∃ any reduction sequence that terminates, the outermost also terminates
- Equivalence:
  - If both orders terminate, they produce the same result
  - Variant of Church-Rosser Theorem
- Cost:
  - Outermost may require more reduction steps
  - But can also bypass unnecessary evals

Redex Example: Same result, more steps

```
square n = n * n
square (3+4)
```

Innermost

```
square (3+4)
=
square 7
=
7 * 7
=
49
```

Outermost

```
square (3+4)
=
(3+4) * (3+4)
=
7 * (3+4)
=
7 * 7
=
49
```

Same result

More steps

Redex Example: Same result, less steps

```
square n = n * n
ignore x = "boondoggle"
```

Innermost

```
ignore square 5
=
ignore 25
=
"boondoggle"
```

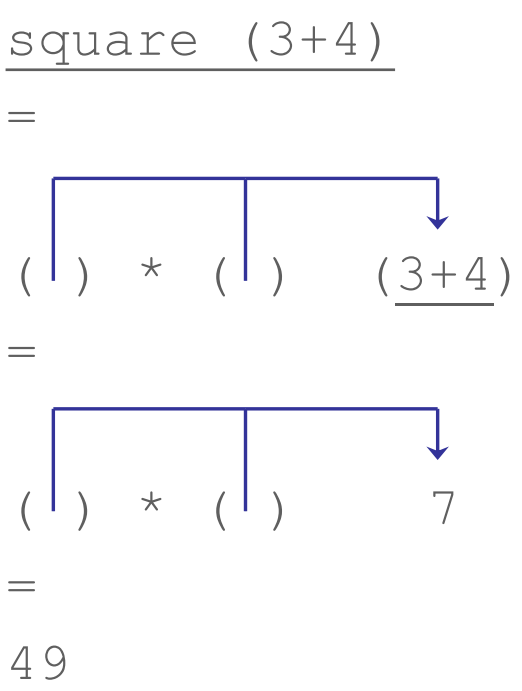
Outermost

```
ignore square 5
=
"boondoggle"
```

Since the result of `ignore` doesn't depend on the value of its argument x, that argument will not be evaluated

Sharing

- Need to avoid repeatedly evaluating the same expression
- Strategy:
  - Each unique expression has a matching data structure (a *Thunk*) on the heap
  - Duplicates of the expression have pointers to the Thunk
  - Thunk replaced with value when evaluation is forced
- An example of memoisation



Redex Termination

```
loop = tail loop
fst (1, loop)
```

Innermost

```
fst (1, loop)
=
fst (1, tail loop)
=
fst (1, tail (tail loop))
=
fst (1, tail (tail (tail
loop)))
…
```

Never terminates

Outermost

```
fst (1, loop)
=
1
```

Terminates in 1 step

Infinite Lists

- Haskell supports potentially infinite lists
  - That are evaluated only to the extent that the context requires
- Example:
  - ```
    ones :: [Int] -- ones produces a list of integers
    ones = 1: ones -- recursively defined
    ```
- Evaluation:
  - ```
    ones = 1: ones
         = 1 : 1 : ones
         = 1 : 1 : 1 : ones
         = ...
    ```

Summary

- Lazy evaluation = outermost reduction + sharing
  - Never requires more steps than innermost reduction, and could require fewer
  - But can use significantly more memory (with Thunks)
- Imperative programming languages generally use strict evaluation
  - Arguments are evaluated before a function is called